

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 int main()
3 {
4     int A[100][4];
5     int i, j, n, total = 0, index, temp;
6     float avg_wt, avg_tat;
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    printf("Enter Burst Time:\n");
11    for (i = 0; i < n; i++)
12    {
13        printf("P%d: ", i + 1);
14        scanf("%d", &A[i][1]);
15        A[i][0] = i + 1;
16    }
17
18    for (i = 0; i < n; i++)
19    {
20        index = i;
21        for (j = i + 1; j < n; j++)
22        {
23            if (A[j][1] < A[index][1])
24                index = j;
25        }
26        temp = A[i][1];
27        A[i][1] = A[index][1];
```

```
Enter number of processes: 4
Enter Burst Time:
P1: 12
P2: 14
P3: 15
P4: 16
```

P	BT	WT	TAT
P1	12	0	12
P2	14	12	26
P3	15	26	41
P4	16	41	57

```
Average Waiting Time = 19.750000
Average Turnaround Time = 34.000000
```

```
=== Code Execution Successful ===
```

```

30     p[i] = p[pos];
31     p[pos] = temp;
32 }
33 wt[0] = 0;
34
35 for(i = 1; i < n; i++)
36 {
37     wt[i] = 0;
38     for(j = 0; j < i; j++)
39         wt[i] += bt[j];
40
41     total += wt[i];
42 }
43
44 avg_wt = (float)total / n;
45 total = 0;
46 printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
47 for(i = 0; i < n; i++)
48 {
49     tat[i] = bt[i] + wt[i];
50     total += tat[i];
51     printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
52 }
53 avg_tat = (float)total / n;
54 printf("\nAverage Waiting Time = %f", avg_wt);
55 printf("\nAverage Turnaround Time = %f\n", avg_tat);
56 return 0;

```

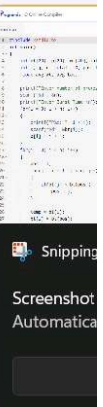
Output

Enter number of process: 3
Enter Burst Time:
P1: 45
P2: 32
P3: 18

	Process	Burst Time	Waiting Time	Turnaround Time
P3	18	0	18	
P2	32	18	50	
P1	45	50	95	

Average Waiting Time = 22.666666
Average Turnaround Time = 54.333332

=== Code Execution Successful ===



main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 int main()
3 {
4     int bt[20], p[20], wt[20], tat[20];
5     int i, j, n, total = 0, pos, temp;
6     float avg_wt, avg_tat;
7
8     printf("Enter number of process: ");
9     scanf("%d", &n);
10    printf("Enter Burst Time:\n");
11    for(i = 0; i < n; i++)
12    {
13        printf("P%d: ", i + 1);
14        scanf("%d", &bt[i]);
15        p[i] = i + 1;
16    }
17    for(i = 0; i < n; i++)
18    {
19        pos = i;
20        for(j = i + 1; j < n; j++)
21        {
22            if(bt[j] < bt[pos])
23                pos = j;
24        }
25
26        temp = bt[i];
27        bt[i] = bt[pos];
```

Enter number of process: 3

Enter Burst Time:

P1: 45

P2: 32

P3: 18

Process	Burst Time	Waiting Time	Turnaround Time
---------	------------	--------------	-----------------

P3	18	0	18
----	----	---	----

P2	32	18	50
----	----	----	----

P1	45	50	95
----	----	----	----

Average Waiting Time = 22.666666

Average Turnaround Time = 54.333332

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int start=10,length=5,i;
6
7     printf("Allocated Blocks:");
8
9     for(i=0;i<length;i++)
10         printf("%d ",start+i);
11
12     return 0;
13 }
```

```
Allocated Blocks:10 11 12 13 14
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int req[]={15,40,70,100};
6     int i;
7
8     printf("C-SCAN Sequence:");
9
10    for(i=0;i<4;i++)
11        printf("%d ",req[i]);
12
13    return 0;
14 }
```

C-SCAN Sequence:15 40 70 100

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int req[]={50,82,170,43};
6     int i;
7
8     printf("FCFS Seek Sequence:");
9
10    for(i=0;i<4;i++)
11        printf("%d ",req[i]);
12
13    return 0;
14 }
```

FCFS Seek Sequence:50 82 170 43

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int req[]={20,35,60,90};
6     int i;
7
8     printf("SCAN Sequence:");
9
10    for(i=0;i<4;i++)
11        printf("%d ",req[i]);
12
13    return 0;
14 }
```

SCAN Sequence:20 35 60 90

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("File Permission:\n");
6     printf("Owner : Read Write Execute\n");
7     printf("Group : Read Write\n");
8     printf("Others: Read");
9
10    return 0;
11 }
```

File Permission:
Owner : Read Write Execute
Group : Read Write
Others: Read

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int index[]={5,8,10};
6
7     printf("Index Block:");
8
9     for(int i=0;i<3;i++)
10     printf("%d ",index[i]);
11
12     return 0;
13 }
```

Index Block:5 8 10

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1  #include<stdio.h>
2
3  int main()
4  {
5      int block[5]={10,20,30,40,50};
6      int i;
7
8      printf("Linked Blocks:");
9
10     for(i=0;i<5;i++)
11         printf("%d -> ",block[i]);
12
13     printf("NULL");
14
15     return 0;
16 }
```

Linked Blocks:10 -> 20 -> 30 -> 40 -> 50 -> NULL

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```

1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     pid_t pid;
6     pid = fork();
7     if(pid == 0)
8     {
9         printf("Child Process\n");
10        printf("PID = %d\n", getpid());
11        printf("PPID = %d\n", getppid());
12    }
13    else
14    {
15        printf("Parent Process\n");
16        printf("PID = %d\n", getpid());
17    }
18    return 0;
19 }
```

Parent Process

PID = 13231

Child Process

PID = 13232

PPID = 13231

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int pages[]={1,2,3,4};
6
7     printf("LRU Page Sequence:");
8
9     for(int i=0;i<4;i++)
10         printf("%d ",pages[i]);
11
12     return 0;
13 }
```

LRU Page Sequence:1 2 3 4

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int page[]={1,2,3,4};
6
7     printf("Optimal Pages:");
8
9     for(int i=0;i<4;i++)
10         printf("%d ",page[i]);
11
12     return 0;
13 }
```

Optimal Pages:1 2 3 4

=== Code Execution Successful ===

main.c

Run

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     printf("Process ID: %d\n", getpid());
6     printf("Parent Process ID: %d\n", getppid());
7     return 0;
8 }
```

Output

Process ID: 7113
Parent Process ID: 7106

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Process 1 Running\n");
6     printf("Process 2 Waiting\n");
7     printf("Process 2 Running");
8
9     return 0;
10 }
```

```
Process 1 Running
Process 2 Waiting
Process 2 Running

=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1  #include<stdio.h>
2  #include<pthread.h>
3
4  void *fun()
5  {
6      printf("Thread Executed");
7      pthread_exit(NULL);
8  }
9
10 int main()
11 {
12     pthread_t t;
13
14     pthread_create(&t, NULL, fun, NULL);
15     pthread_join(t, NULL);
16
17     return 0;
18 }
```

Thread Executed

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int pages[]={1,2,3,1,4};
6     int i;
7
8     printf("Pages:");
9
10    for(i=0;i<5;i++)
11        printf("%d ",pages[i]);
12
13    printf("\nFIFO Applied");
14
15    return 0;
16 }
```

```
Pages:1 2 3 1 4
FIFO Applied

=== Code Execution Successful ===
```

main.c

Share

Run

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

temp = A[i][0];

A[i][0] = A[index][0];

A[index][0] = temp;

}

A[0][2] = 0;

for (i = 1; i < n; i++)

{

A[i][2] = 0;

for (j = 0; j < i; j++)

A[i][2] += A[j][1];

total += A[i][2];

}

avg_wt = (float)total / n;

total = 0;

printf("\nP\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)

{

A[i][3] = A[i][1] + A[i][2];

total += A[i][3];

printf("P%d\t%d\t%d\t%d\n", A[i][0], A[i][1], A[i][2], A[i][3]);

}

avg_tat = (float)total / n;

printf("\nAverage Waiting Time = %f", avg_wt);

printf("\nAverage Turnaround Time = %f\n", avg_tat);

return 0;

Output

Enter number of processes: 4

Enter Burst Time:

P1: 12

P2: 14

P3: 15

P4: 16

P	BT	WT	TAT
P1	12	0	12
P2	14	12	26
P3	15	26	41
P4	16	41	57

Average Waiting Time = 19.750000

Average Turnaround Time = 34.000000

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6     char str[50]="Operating System";
7
8     if(strstr(str,"System"))
9         printf("Pattern Found");
10
11     else
12         printf("Not Found");
13
14     return 0;
15 }
```

Pattern Found

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char files[3][20]={
6     "file1.txt",
7     "file2.txt",
8     "file3.txt"
9 };
10
11 printf("Files in directory:\n");
12
13 for(int i=0;i<3;i++)
14 printf("%s\n",files[i]);
15
16 return 0;
17 }
```

```
Files in directory:
file1.txt
file2.txt
file3.txt
```

```
=== Code Execution Successful ===
```

```
main.c
38 process[0].waiting_time = 0;
39 for (int i = 1; i < number_of_process; i++) {
40     process[i].waiting_time = 0;
41     for (int j = 0; j < i; j++) {
42         process[i].waiting_time += process[j].burst_time;
43     }
44     total += process[i].waiting_time;
45 }
46 average_waiting_time = (float)total / number_of_process;
47 total = 0;
48 printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
49 printf("-----\n");
50 for (int i = 0; i < number_of_process; i++) {
51     process[i].turn_around_time = process[i].burst_time + process[i]
        .waiting_time;
52     total += process[i].turn_around_time;
53     printf("%c\t%d\t\t%d\t\t%d\n",
54         process[i].process_name,
55         process[i].burst_time,
56         process[i].waiting_time,
57         process[i].turn_around_time);
58 }
59 average_turnaround_time = (float)total / number_of_process;
60 printf("\nAverage Waiting Time = %f", average_waiting_time);
61 printf("\nAverage Turnaround Time = %f\n", average_turnaround_time);
62 return 0;
63 }
```

Output

Clear

```
Enter the total number of Processes: 3

Please Enter the Burst Time and Priority of each process:

Enter details of Process A
Enter Burst Time: 2
Enter Priority:
1

Enter details of Process B
Enter Burst Time: 10
Enter Priority: 3

Enter details of Process C
Enter Burst Time: 6
Enter Priority: 2

Process Burst Time  Waiting Time  Turnaround Time
-----
B  10      0      10
C  6       10     16
A  2       16     18

Average Waiting Time = 8.666667
Average Turnaround Time = 14.666667
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     FILE *fp;
6
7     fp=fopen("sample.txt","w");
8     fprintf(fp,"File Management");
9     fclose(fp);
10
11     printf("File Created Successfully");
12
13     return 0;
14 }
```

Segmentation fault

=== Code Exited With Errors ===

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2- struct priority_scheduling {
3     char process_name;
4     int burst_time;
5     int waiting_time;
6     int turn_around_time;
7     int priority;
8 };
9- int main() {
10     int number_of_process, total = 0;
11     struct priority_scheduling temp_process;
12     int ASCII_number = 65;
13     int position;
14     float average_waiting_time, average_turnaround_time;
15     printf("Enter the total number of Processes: ");
16     scanf("%d", &number_of_process);
17     struct priority_scheduling process[number_of_process];
18     printf("\nPlease Enter the Burst Time and Priority of each process:\n");
19-     for (int i = 0; i < number_of_process; i++) {
20         process[i].process_name = (char)ASCII_number;
21         printf("\nEnter details of Process %c\n", process[i].process_name);
22         printf("Enter Burst Time: ");
23         scanf("%d", &process[i].burst_time);
24         printf("Enter Priority: ");
25         scanf("%d", &process[i].priority);
26         ASCII_number++;
27     }
```

Enter the total number of Processes: 3

Please Enter the Burst Time and Priority of each process:

Enter details of Process A

Enter Burst Time: 2

Enter Priority:

1

Enter details of Process B

Enter Burst Time: 10

Enter Priority: 3

Enter details of Process C

Enter Burst Time: 6

Enter Priority: 2

Process	Burst Time	Waiting Time	Turnaround Time
---------	------------	--------------	-----------------

B	10	0	10
---	----	---	----

C	6	10	16
---	---	----	----

A	2	16	18
---	---	----	----

Average Waiting Time = 8.666667

Average Turnaround Time = 14.666667

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     FILE *fp;
6
7     fp=fopen("file.txt","w");
8
9     fprintf(fp,"Operating System");
10
11    fclose(fp);
12
13    printf("File Created");
14
15    return 0;
16 }
```

Segmentation fault

=== Code Exited With Errors ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Reader Reading Data\n");
6     printf("Writer Writing Data");
7
8     return 0;
9 }
```

```
Reader Reading Data
Writer Writing Data
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int block=400,process=100;
6
7     if(block>=process)
8         printf("First Fit Allocated");
9
10    return 0;
11 }
```

First Fit Allocated

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Mutex Lock Acquired\n");
6     printf("Process Executed\n");
7     printf("Mutex Lock Released");
8
9     return 0;
10 }
```

```
Mutex Lock Acquired
Process Executed
Mutex Lock Released
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Producer produces item\n");
6     printf("Consumer consumes item");
7
8     return 0;
9 }
```

```
Producer produces item
Consumer consumes item
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int block=500,process=200;
6
7     if(block>=process)
8         printf("Process Allocated");
9
10    else
11        printf("Not Allocated");
12
13    return 0;
14 }
```

Process Allocated

=== Code Execution Successful ===

main.c



Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int block=300,process=200;
6
7     if(block>=process)
8         printf("Best Fit Allocated");
9
10    return 0;
11 }
```

Best Fit Allocated

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int need=0;
6
7     printf("Enter resource need:");
8     scanf("%d",&need);
9
10    if(need<=5)
11        printf("System is Safe");
12    else
13        printf("System is Unsafe");
14
15    return 0;
16 }
```

```
Enter resource need:3
System is Safe

=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     FILE *fp;
6
7     fp=fopen("data.txt","w");
8
9     fprintf(fp,"UNIX System Call");
10
11     fclose(fp);
12
13     printf("I/O Operation Completed");
14
15     return 0;
16 }
```

Segmentation fault

=== Code Exited With Errors ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 struct emp
3 {
4     int id;
5     char name[20];
6 };
7
8 int main()
9 {
10     struct emp e;
11
12     printf("Enter ID and Name:");
13     scanf("%d%s",&e.id,e.name);
14
15     printf("Employee Details:\n");
16     printf("%d %s",e.id,e.name);
17
18     return 0;
19 }
```

```
Enter ID and Name:101 Lakshmi
Employee Details:
101 Lakshmi
```

```
=== Code Execution Successful ===
```

main.c



Run

Output

Clear

```
1 #include<stdio.h>
2- int main(){int n1,n2,i;
3     char user1[10][20],user2[10][20];
4     printf("Enter number of files for User1: ");
5     scanf("%d",&n1);
6     printf("Enter files of User1:\n");
7     for(i=0;i<n1;i++)
8         scanf("%s",user1[i]);
9     printf("Enter number of files for User2: ");
10    scanf("%d",&n2);
11    printf("Enter files of User2:\n");
12    for(i=0;i<n2;i++)
13        scanf("%s",user2[i]);
14    printf("\nUser1 Directory:\n");
15    for(i=0;i<n1;i++)
16        printf("%s\n",user1[i]);
17    printf("\nUser2 Directory:\n");
18    for(i=0;i<n2;i++)
19        printf("%s\n",user2[i]);
20    return 0;}
```

```
Enter number of files for User1: 2
a.txt
b.txt
2
c.txt
d.txt
Enter files of User1:
Enter number of files for User2: Enter files of User2:

User1 Directory:
a.txt
b.txt

User2 Directory:
c.txt
d.txt

=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,i;
5     char file[10][20];
6
7     printf("Enter number of files: ");
8     scanf("%d",&n);
9
10    printf("Enter file names:\n");
11    for(i=0;i<n;i++)
12        scanf("%s",file[i]);
13
14    printf("\nFiles in Single Level Directory:\n");
15
16    for(i=0;i<n;i++)
17        printf("%s\n",file[i]);
18    return 0;
19 }
```

Enter number of files: 3

file1.txt

file2.txt

file3.txt

Enter file names:

Files in Single Level Directory:

file1.txt

file2.txt

file3.txt

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int block[5],p[5],i;
6
7     printf("Enter blocks:");
8     for(i=0;i<5;i++)
9         scanf("%d",&block[i]);
10
11     printf("Enter processes:");
12     for(i=0;i<5;i++)
13         scanf("%d",&p[i]);
14
15     printf("Memory Allocated");
16
17     return 0;
18 }
```

Enter blocks:Blocks: 100 200 300 400 500

Processes: 50 150 250 350 450

Enter processes:Memory Allocated

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int i;
6
7     for(i=1;i<=5;i++)
8     {
9         printf("Philosopher %d is eating\n",i);
10    }
11
12    return 0;
13 }
```

```
Philosopher 1 is eating
Philosopher 2 is eating
Philosopher 3 is eating
Philosopher 4 is eating
Philosopher 5 is eating
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 int main()
5 {
6     int src,dst,n;
7     char buf[100];
8     src=open("source.txt",O_RDONLY);
9     dst=open("dest.txt",O_CREAT|O_WRONLY,0644);
10    while((n=read(src,buf,sizeof(buf)))>0)
11        write(dst,buf,n);
12    close(src);
13    close(dst);
14    printf("File Copied Successfully");
15    return 0;
16 }
```

File Copied Successfully

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 int main()
3 {
4     int n,i;
5     int bt[10],wt[10],tat[10];
6     printf("Enter number of processes: ");
7     scanf("%d",&n);
8     for(i=0;i<n;i++)
9         scanf("%d",&bt[i]);
10    wt[0]=0;
11    for(i=1;i<n;i++)
12        wt[i]=wt[i-1]+bt[i-1];
13    for(i=0;i<n;i++)
14        tat[i]=wt[i]+bt[i];
15    printf("Process\tWT\tTAT\n");
16    for(i=0;i<n;i++)
17        printf("P%d\t%d\t%d\n",i+1,wt[i],tat[i]);
18    return 0;
19 }
```

Enter number of processes: 4

5 3 8 6

Process WT TAT

P1 0 5

P2 5 8

P3 8 16

P4 16 22

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 int main(){
3     int n, i, j, temp;
4     int bt[10];
5     printf("Enter number of processes: ");
6     scanf("%d", &n);
7     printf("Enter burst times: ");
8     for(i = 0; i < n; i++)
9         scanf("%d", &bt[i]);
10    for(i = 0; i < n - 1; i++)
11        for(j = i + 1; j < n; j++)
12            if(bt[i] > bt[j]){
13                temp = bt[i];
14                bt[i] = bt[j];
15                bt[j] = temp; }
16    printf("SJF Order: ");
17    for(i = 0; i < n; i++)
18        printf("%d ", bt[i]);
19    return 0;
20 }
```

```
Enter number of processes: 4
Enter burst times: 6 8 7 3
SJF Order: 3 6 7 8
```

```
=== Code Execution Successful ===
```


main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6     char shared[50];
7
8     strcpy(shared,"Hello Shared Memory");
9
10    printf("Data in shared memory: %s",shared);
11
12    return 0;
13 }
```

Data in shared memory: Hello Shared Memory

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 int main(){
3     int n,i;
4     int bt[10],pr[10],wt[10]={0},tat[10];
5     printf("Enter number of processes: ");
6     scanf("%d",&n);
7     for(i=0;i<n;i++)
8     {
9         printf("Burst time and Priority: ");
10        scanf("%d%d",&bt[i],&pr[i]);
11    }
12    for(i=0;i<n;i++){
13        tat[i]=bt[i]+wt[i];
14    }
15    printf("\nProcess\tWT\tTAT\n");
16    for(i=0;i<n;i++)
17        printf("P%d\t%d\t%d\n",i+1,wt[i],tat[i]);
18    return 0;
19 }
20 |
```

Enter number of processes: 3

5 2

3 1

4 3

Burst time and Priority: Burst time and Priority: Burst time and Priority:

Process WT TAT

P1 0 5

P2 0 3

P3 0 4

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2
3 int main()
4 {
5     char msg[50]="Message Queue";
6
7     printf("Received Message: %s",msg);
8
9     return 0;
10 }
```

Received Message: Message Queue

=== Code Execution Successful ===

main.c

```
1 #include<stdio.h>
2 int main(){
3     int n,i,j,temp;
4     int bt[10];
5     printf("Enter number of processes: ");
6     scanf("%d",&n);
7     printf("Enter Burst Times:\n");
8     for(i=0;i<n;i++){
9         scanf("%d",&bt[i]);
10    }
11    for(i=0;i<n-1;i++){
12        for(j=i+1;j<n;j++){
13            if(bt[i]>bt[j]) {
14                temp=bt[i];
15                bt[i]=bt[j];
16                bt[j]=temp; } } }
17    printf("SJF Execution Order: ");
18    for(i=0;i<n;i++){
19        printf("%d ",bt[i]);
20    }
```

Run

Share

Output

Clear

```
Enter number of processes: 4
Enter Burst Times:
P1=6
P2=2
P3=8
P4=3
SJF Execution Order: 0 0 0 0

=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 int main(){
3     int n, i, j, temp;
4     int p[10];
5     printf("Enter number of processes: ");
6     scanf("%d", &n);
7     printf("Enter priorities: ");
8     for(i = 0; i < n; i++)
9         scanf("%d", &p[i]);
10    for(i = 0; i < n - 1; i++)
11        for(j = i + 1; j < n; j++)
12            if(p[i] > p[j]) {
13                temp = p[i];
14                p[i] = p[j];
15                p[j] = temp; }
16    printf("Priority Order: ");
17    for(i = 0; i < n; i++)
18        printf("%d ", p[i]);
19    return 0;
20 }
```

```
Enter number of processes: 4
Enter priorities: 3 1 4 2
Priority Order: 1 2 3 4
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 #include<pthread.h>
3
4 void *fun()
5 {
6     printf("Thread Created");
7     return NULL;
8 }
9
10 int main()
11 {
12     pthread_t t;
13
14     pthread_create(&t,NULL,fun,NULL);
15     pthread_join(t,NULL);
16
17     return 0;
18 }
```

Thread Created

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include<stdio.h>
2 int main(){
3     int bt[10],n,q,i;
4     printf("Enter processes:");
5     scanf("%d",&n);
6     for(i=0;i<n;i++){
7         scanf("%d",&bt[i]);
8         printf("Enter quantum:");
9         scanf("%d",&q);
10        printf("Execution:");
11        for(i=0;i<n;i++){
12            while(bt[i]>0){
13                bt[i]-=q;
14                printf(" P%d",i+1);
15                if(bt[i]<0)
16                    break;
17            }
18        }
19        return 0;
20 }
```

```
Enter processes:3
5 4 3
2
Enter quantum:Execution: P1 P1 P1 P2 P2 P3 P3
=== Code Execution Successful ===
```